06. 퍼스널리티와 컨텍스트

Hermes Agent는 기본 성격 그대로 사용해도 충분히 유능합니다. 하지만 팀의 코딩 컨벤션에 맞게 답변하거나, 한국어로만 대화하거나, 특정 프로젝트의 규칙을 자동으로 따르게 하고 싶다면 어떻게 해야 할까요? 이 장에서는 Hermes Agent의 성격과 작동 방식을 커스터마이징하는 두 가지 핵심 메커니즘을 다룹니다. SOUL.md는 에이전트의 영구적인 성격을, AGENTS.md는 프로젝트별 작업 지침을 정의합니다.

이 장에서 다루는 내용

- SOUL.md로 성격 정의
- AGENTS.md로 프로젝트 지침 설정
- 실습: 나만의 에이전트 만들기

06-1. SOUL.md로 성격 정의

06-1. SOUL.md로 성격 정의

(콘텐츠를 불러올 수 없습니다)

06-2. AGENTS.md로 프로젝트 지침 설정

> ⚠️ ***부분 검증**: 이 페이지는 v0.16.0 소스 코드(`agent/prompt_builder.py` 의 컨텍스트 파일 로딩)로 확인했습니다. 프로젝트 컨텍스트는 우선순위에 따라 한 종류만 로딩되고, AGENTS.md는 작업 디렉터리 최상위 파일 하나만 읽습니다(과거의 재귀적 계층 탐색은 제거됨).*

- SOUL.md와의 차이
- 로딩 방식
- 프롬프트 내 위치
- 작성 예시
- 크기 제한
- 보안 스캔
- 호환 파일
- BOOT.md — 게이트웨이 시작 시 자동 실행

AGENTS.md는 **프로젝트별 작업 지침**을 정의하는 파일입니다. 프로젝트 디렉터리에 위치하며, 에이전트가 해당 디렉터리에서 작업할 때 자동으로 로딩됩니다.

SOUL.md와의 차이

공식 가이드의 표현을 빌리면: "어디에나 따라다녀야 할 것은 SOUL.md에, 프로젝트에 속하는 것은 AGENTS.md에."

- **SOUL.md**: `~/.hermes/SOUL.md` 에 위치. 전역적. 에이전트의 **정체성** (톤, 성격, 대화 방식)
- **AGENTS.md**: 프로젝트 디렉터리에 위치. 지역적. 프로젝트의 **작업 지침** (코딩 규칙, 기술 스택, 파일 구조)

로딩 방식

AGENTS.md는 **현재 작업 디렉터리(cwd) 최상위의 파일 하나**만 읽습니다. 예전에는 하위 디렉터리까지 재귀적으로 모든 AGENTS.md를 모아 깊이순으로 결합했지만, 그 방식은 제거됐습니다. 지금은 작업 디렉터리의 AGENTS.md 한 개만 로딩되고, 하위 폴더의 AGENTS.md는 자동으로 합쳐지지 않습니다.

또한 프로젝트 컨텍스트 파일은 **우선순위에 따라 처음 발견한 한 종류만** 로딩됩니다.

1. `.hermes.md` / `HERMES.md` (git root까지 상위 탐색)
2. `AGENTS.md` / `agents.md` (cwd만)

3. `CLAUDE.md` / `claude.md` (cwd만)
4. `.cursorrules` / `.cursor/rules/*.mdc` (cwd만)

즉 `.hermes.md`가 있으면 AGENTS.md는 읽지 않습니다. SOUL.md는 이 우선순위와 별개로 정체성 슬롯에 항상 포함됩니다.

프롬프트 내 위치

AGENTS.md는 시스템 프롬프트에서 SOUL.md보다 뒤쪽(슬롯 7)에 위치합니다.

```
슬롯 1: SOUL.md (정체성)
슬롯 2: 도구 사용 가이드
슬롯 3: Honcho (활성 시)
슬롯 4: 게이트웨이 시스템 메시지
슬롯 5: 메모리 + 사용자 프로필
슬롯 6: 스킬 인덱스
슬롯 7: 컨텍스트 파일 (AGENTS.md, .cursorrules 등) ← 여기
슬롯 8: 타임스탬프 + 모델 정보
슬롯 9: 플랫폼 힌트
```

작성 예시

```
# AGENTS.md

## 프로젝트 구조
- Python 3.11, FastAPI 기반 백엔드
- React 18 프런트엔드
- PostgreSQL 데이터베이스

## 코딩 규칙
- Python 코드는 ruff로 포맷팅
- 타입 힌트 필수
- 테스트는 pytest 사용
- 커밋 메시지는 Conventional Commits 형식

## 환경
- 개발 서버: localhost:8000
- DB: localhost:5432/mydb
```

크기 제한

컨텍스트 파일은 소스별로 각각 **최대 20,000자**로 제한됩니다(합산이 아니라 파일 단위). AGENTS.md는 현재 작업 디렉터리 최상위의 파일 하나만 읽습니다. 초과 시 스마트 절삭(앞 70% + 뒤 20%)이 적용됩니다.

AGENTS.md 하나가 2만 자를 넘으면 가운데가 잘리므로, 길어질 때는 핵심 규칙만 남기는 것이 좋습니다.

보안 스캔

AGENTS.md는 로딩 시 SOUL.md와 동일한 보안 스캔을 거칩니다.

- 프롬프트 인젝션 패턴 10종 ("이전 지시사항을 무시해라" 등)
- 보이지 않는 유니코드 문자 10종 (제로 폭 공백, 양방향 오버라이드 등)
- 자격증명 유출 시도

탐지되면 해당 파일 전체가 `[BLOCKED: ...]` 메시지로 대체됩니다. 공유 저장소에서 다른 사람이 작성한 AGENTS.md는 내용을 확인한 뒤 사용하세요.

호환 파일

Hermes Agent는 AGENTS.md 외에도 다음 파일을 자동으로 로딩합니다.

파일	탐색 범위	비고
`.cursorrules`	CWD만	Cursor IDE 규칙 파일
`.cursor/rules/*.mdc`	CWD만	Cursor IDE 규칙 디렉터리
`.hermes.md` / `HERMES.md`	CWD → git root까지 상위 탐색	YAML 프런트매터 자동 제거

`.hermes.md`는 공식 문서에 기재되어 있지 않지만 소스 코드에서 확인된 기능입니다. git root까지 상위 디렉터리를 탐색하며 처음 발견한 파일을 사용합니다.

BOOT.md — 게이트웨이 시작 시 자동 실행

`~/.hermes/BOOT.md` 파일을 작성해 두면, 게이트웨이가 시작될 때 이 파일의 내용을 첫 번째 프롬프트로 자동 실행합니다. 매번 수동으로 초기 지시를 내릴 필요 없이, 시작과 동시에 특정 작업을 수행하도록 설정할 수 있습니다.

예를 들어 "서버 상태를 점검하고 이상이 있으면 Telegram으로 알려줘"라는 내용을 BOOT.md에 넣어 두면, 게이트웨이가 재시작될 때마다 자동으로 실행됩니다.

06-3. 실습: 나만의 에이전트 만들기

> ⚠️ ***미검증**: 실습 내용은 소스 코드 분석과 커뮤니티 사례를 바탕으로 설계했으며, 실제 테스트는 추후 진행 예정입니다.*

- 실습 전제와 상태 관리
- 실습 1: SOUL.md로 행동 규칙 정의
  - 기본 상태 확인
  - 커스텀 SOUL.md 작성
  - 변화 확인
  - 관찰 포인트
- 실습 2: /personality로 톤 전환
  - SOUL.md와 /personality가 충돌하면?
  - 초기화
  - 관찰 포인트
- 실습 3: AGENTS.md로 프로젝트별 지침 설정
  - 관찰 포인트
  - 세 파일의 역할 정리

이 절에서는 SOUL.md, `/personality`, AGENTS.md를 직접 설정하며 각각의 역할 차이를 체험합니다.

실습 전제와 상태 관리

세 실습은 모두 에이전트의 시스템 프롬프트나 작업 디렉터리 상태를 바꾸므로, 시작 전·끝낸 뒤에 다음 사항을 확인하면 좋습니다.

- **현재 SOUL.md 백업**: 실습 1에서 SOUL.md를 직접 덮어쓰므로 기존 내용을 별도 경로에 복사해 두세요. `cp ~/.hermes/SOUL.md ~/.hermes/SOUL.md.bak`
- **현재 `/personality` 확인**: `/personality`는 config에 저장돼 다음 세션에도 유지됩니다. 실습 전 현재 값을 적어 두고, 실습 후 원래 값으로 되돌리거나 `/personality none`으로 초기화합니다.
- **새 세션으로 검증**: SOUL.md·AGENTS.md 변경은 새 세션부터 반영됩니다. 변경 후에는 항상 `/new`로 새 세션을 열어 확인합니다.
- **권장 순서**: 실습 1 → 2 → 3 순으로 진행합니다. 실습 1·2는 같은 SOUL.md를 공유하므로 차례로 따라가면 자연스럽게 이어집니다. 실습 3은 별도 프로젝트 디렉터리에서 진행하므로 앞 실습 상태와 독립적입니다.

실습 1: SOUL.md로 행동 규칙 정의

SOUL.md는 에이전트의 **영구적 정체성**입니다. "어떤 성격인지"보다 "어떻게 행동할지"를 정의할 때 효과적입니다.

기본 상태 확인

먼저 기본 SOUL.md 상태에서 에이전트의 행동을 관찰합니다.

```
> 현재 디렉터리의 파일 목록을 정리해줘
```

기본 템플릿에서는 영어 중심, 간결한 응답으로 결과를 보여줍니다.

커스텀 SOUL.md 작성

`~/.hermes/SOUL.md`를 직접 수정합니다. 성격 서술("나는 친절한 AI야") 대신 **행동 규칙**을 적습니다.

```
# 나의 AI 비서

나는 실용적이고 직접적인 한국어 AI 비서야.

## 행동 규칙
- 한국어로 대화한다
- 동료처럼 대등하게 말한다
- 모르는 것은 모른다고 솔직히 말한다
- "물론이죠!", "좋은 질문입니다!" 같은 빈말 금지

## 작업 방식
- 작업 전에 계획을 한 줄로 요약한다
- 위험한 명령은 실행 전에 확인한다
- 에러 발생 시 원인 분석을 먼저 한다
- 여러 방법이 있으면 장단점을 비교한다

## 응답 형식
- 핵심을 먼저, 부연은 뒤에
- 코드 블록에는 반드시 주석을 단다
- 표나 목록으로 정리할 수 있으면 산문보다 표를 쓴다
```

변화 확인

SOUL.md는 세션 시작 시 로딩되므로, 반드시 새 세션을 시작해야 합니다.

```
> /new
> 현재 디렉터리의 파일 목록을 정리해줘
```

같은 요청인데 응답이 달라졌는지 확인합니다. 한국어로 답하는지, 계획을 먼저 요약하는지, 표 형식을 쓰는지 등을 관찰하세요.

이어서 행동 규칙이 실제로 작동하는지 테스트합니다.

```
> 양자 컴퓨팅의 원리를 설명해줘
```

"모르는 것은 모른다고 솔직히 말한다" 규칙과 "빈말 금지" 규칙이 지켜지는지 관찰합니다. 과장 없이 아는 범위를 밝히는지, 또는 장황한 인사 없이 바로 설명에 들어가는지 확인하세요.

관찰 포인트

- SOUL.md를 수정한 뒤 `/new`로 새 세션을 시작해야 변경이 반영됩니다
- SOUL.md가 기본 정체성을 **완전히 대체**합니다. "도움이 되는 AI"라는 기본 지시도 사라지므로, 원하는 특성은 빠짐없이 명시하세요
- 30~80줄이 적정 범위입니다. 너무 길면 모델이 일부를 무시할 수 있습니다
- "친절한 AI야"보다 "빈말 금지", "계획을 먼저 요약" 같은 **구체적 행동 규칙**이 모델에 훨씬 잘 전달됩니다

실습 2: /personality로 톤 전환

`/personality`는 SOUL.md를 건드리지 않고 **톤만 바꾸는 오버레이**입니다. 시스템 프롬프트 맨 끝에 추가되므로, SOUL.md의 행동 규칙은 유지한 채 어조만 전환됩니다.

실습 1에서 작성한 SOUL.md를 그대로 둔 채 진행합니다.

```
> /personality concise
> Python으로 피보나치 함수를 작성해줘
```

```
> /personality teacher
> Python으로 피보나치 함수를 작성해줘
```

같은 질문에 대해 답변 길이와 설명 깊이가 어떻게 달라지는지 비교합니다. 두 경우 모두 SOUL.md의 행동 규칙(한국어, 코드 주석, 빈말 금지)은 유지되는지도 확인하세요.

SOUL.md와 /personality가 충돌하면?

SOUL.md에 "한국어로 대화한다"고 적었는데 `/personality pirate`를 적용하면 어떻게 될까요?

```
> /personality pirate
> 오늘 날씨가 어때?
```

SOUL.md(슬롯 1)와 `/personality`(맨 끝)가 동시에 적용되며, 두 지시가 충돌하면 뒤에 오는 `/personality` 쪽이 더 강하게 작용합니다. 에이전트가 영어 해적 말투로 전환하는지, 한국어 해적 말투를 시도하는지 관찰해 보세요.

초기화

실험이 끝나면 반드시 초기화합니다. `/personality`는 config에 저장되어 다음 세션에도 유지되기 때문입니다.

```
> /personality none
```

관찰 포인트

- SOUL.md = 영구적 행동 규칙, `/personality` = 일시적 톤 전환. 역할이 다릅니다
- `/personality`는 SOUL.md를 대체하지 않고, 시스템 프롬프트 **끝에 추가**됩니다
- 충돌 시 `/personality`가 우세하지만, 행동 규칙(계획 요약, 주석 등)은 대체로 유지됩니다
- config에 저장되므로 실험 후 `/personality none`을 잊지 마세요

실습 3: AGENTS.md로 프로젝트별 지침 설정

AGENTS.md는 **프로젝트 디렉터리에 두는 작업 지침**입니다. SOUL.md가 "이 에이전트가 누구인지"라면, AGENTS.md는 "이 프로젝트에서 어떻게 일할지"입니다.

```
mkdir -p ~/test-project
```

`~/test-project/AGENTS.md`를 다음 내용으로 작성합니다.

```
# 테스트 프로젝트

## 코딩 규칙
- Python 3.11 사용
- 변수명은 snake_case
- 모든 함수에 docstring 작성
- print 대신 logging 모듈 사용
- 타입 힌트 필수
```

해당 디렉터리에서 Hermes Agent를 실행합니다.

```
cd ~/test-project
hermes
```

에이전트에게 코드를 요청하고, AGENTS.md의 규칙을 따르는지 확인합니다.

```
> 간단한 HTTP 서버 코드를 작성해줘
```

관찰 포인트

- logging 모듈을 사용하는지 (print 대신)
- snake_case를 따르는지
- docstring이 있는지
- 타입 힌트가 있는지
- AGENTS.md가 없는 디렉터리에서 같은 요청을 하면 어떻게 다른지도 비교해 보세요

세 파일의 역할 정리

	SOUL.md	/personality	AGENTS.md
위치	`~/.hermes/SOUL.md`	명령어 또는 config	프로젝트 디렉터리
범위	모든 세션, 모든 프로젝트	세션 (config 저장 시 영구)	해당 프로젝트만
용도	행동 규칙, 의사결정 방식	톤/어조 전환	코딩 규칙, 프로젝트 구조
비유	사람의 가치관	상황에 따른 말투	팀의 코딩 컨벤션